

The Reconstruction Threshold for the Ferromagnetic Four-State Potts Model

OpenAI

October 5, 2026

Abstract

We establish the exact Kesten–Stigum reconstruction threshold for the ferromagnetic four-state Potts broadcast model on every regular d -ary tree with $d \geq 2$ and every Poisson Galton–Watson tree of mean $d > 0$. Reconstruction occurs exactly when $d\lambda^2 > 1$; we prove nonreconstruction at and below the threshold, including equality. In the Poisson model the whole tree is observed and the reconstruction advantage is averaged without conditioning on survival. The proof uses reproducible exact-arithmetic verification of polynomial inequalities.

1 Introduction

In a broadcast process on a tree, each vertex transmits a noisy copy of its spin to its children. The reconstruction problem asks whether the spins far from the root retain information about its initial spin. For the symmetric four-state channel, the linear second-moment threshold has long been expected to be the exact reconstruction threshold in the ferromagnetic regime. We prove the nonreconstruction direction at and below this threshold for all regular and Poisson branching parameters.

1.1 Model and main result

Write $[4] = \{1, 2, 3, 4\}$. The root spin σ_ρ is uniform on $[4]$. For a parameter $\lambda \in [0, 1]$, each edge independently transmits the parent spin through the channel

$$P_\lambda(j \mid i) = \lambda \mathbf{1}_{\{i=j\}} + \frac{1-\lambda}{4}, \quad i, j \in [4]. \quad (1.1)$$

The parameter λ is the nontrivial eigenvalue of the channel; its nonnegativity specifies the ferromagnetic regime.

We consider the rooted d -ary tree, where every vertex has exactly d children, and a Galton–Watson tree with independent $\text{Poisson}(d)$ offspring counts. In the latter case the tree is sampled independently of the root spin and the randomness used by the edge channels. The observer is given the entire rooted tree T and the spins σ_{L_n} at level $L_n = \{v : \text{dist}(v, \rho) = n\}$. Let

$$a_n(d, \lambda) = \mathbb{E} \left[\frac{1}{2} \sum_{i=1}^4 \left| \Pr(\sigma_\rho = i \mid T, \sigma_{L_n}) - \frac{1}{4} \right| \right]. \quad (1.2)$$

The tree observation in the Galton–Watson model is unlabelled, the level observation is empty when L_n is empty, and the expectation is not conditioned on survival. Nonreconstruction means that $a_n(d, \lambda)$ tends to zero.

Theorem 1.1. *For the channel (1.1), if $d\lambda^2 \leq 1$, then*

$$\lim_{n \rightarrow \infty} a_n(d, \lambda) = 0$$

in each of the following models:

1. *the rooted d -ary tree, for every integer $d \geq 2$;*
2. *the $\text{Poisson}(d)$ Galton–Watson tree, for every real $d > 0$.*

In particular, the assertion includes $d\lambda^2 = 1$.

Here d denotes the number of children or its mean, not the total undirected degree. The known Kesten–Stigum reconstruction direction for $d\lambda^2 > 1$ makes Theorem 1.1 an exact threshold statement [6, 11, 8]. For Poisson trees, the branching-number reconstruction criterion applies on survival, where the branching number equals d ; survival has positive probability when $d\lambda^2 > 1$. The theorem is computer assisted: all probabilistic arguments are proved below, and the remaining polynomial inequalities are verified by the complete exact-arithmetic programs in Appendix A.

1.2 History and significance

The Kesten–Stigum criterion originates in the second-moment analysis of multitype branching processes [6]. In tree broadcasting it gives the reconstruction direction above $d\lambda^2 = 1$, but the corresponding nonreconstruction assertion depends on the channel. For binary symmetric channels, Bleher, Ruiz, and Zagrebnev established sharpness on regular trees [2]; Evans, Kenyon, Peres, and Schulman developed the general-tree theory [3]. Mossel showed that the spectral criterion need not be necessary for other channels [10]. For Potts channels, this distinction becomes decisive as the number of states increases. Mézard and Montanari proposed threshold coincidence for small alphabets and discussed its extension to all regular degrees [9, Section 8]. Sly proved sharpness for the three-state model on regular trees of sufficiently large degree and established failure of sharpness for $q \geq 5$ [14]. The four-state case lies at the boundary between these behaviors.

Mossel, Sly, and Sohn proved nonreconstruction, including the critical point, for the three- and four-state models on Galton–Watson trees of sufficiently large mean degree under hypotheses that cover both regular and Poisson offspring [12]. Theorem 1.1 removes the large-degree restriction for the ferromagnetic four-state model in these two families. Cavity expansions and numerical investigations by Ricci-Tersenghi, Semerjian, and Zdeborová also support the four-state ferromagnetic threshold [13]; those predictions are distinct from a rigorous all-degree nonreconstruction theorem. The restriction $\lambda \geq 0$ matters: the four-state antiferromagnetic model can reconstruct below the Kesten–Stigum threshold [12].

1.3 The proof mechanism

The key object is the distribution of a posterior probability vector $p = (p_1, p_2, p_3, p_4)$, not an individual vector. Define its quadratic information by

$$A(p) = 4 \sum_{i=1}^4 p_i^2 - 1.$$

A channel step multiplies $\mathbb{E}A$ by λ^2 . The problem is to control the nonlinear combination of the observations arriving from different children. We construct a degree-six polynomial H for which

the additional constraint $\mathbb{E}H(p) \geq 0$ is preserved by both channel steps and the combination of observations. Two explicit polynomial inequalities then make EA subadditive under combination, with a strictly positive cubic saving when two informative branches are present.

Writing m_n for the expected quadratic information at depth n , this gives, for $\lambda > 0$, a recursion of the form

$$m_{n+1} \leq d\lambda^2 m_n - cm_n^3, \quad c = c(d, \lambda) > 0.$$

The cubic term is essential at equality: it forces $m_n \rightarrow 0$ even when the linear coefficient equals one. The constraint on the posterior law is linear, so it also survives mixtures over the observed offspring count. This is what allows the same pairwise inequalities to treat the Poisson and regular models.

Expected functionals of posterior distributions are central to earlier entropy and stochastic Lyapunov methods, including the work of Formentin and Külske [4]. Rigorous computational methods for nonreconstruction include Bhatnagar and Maneva’s survey method [1] and Gu’s population-dynamics method for binary Poisson hypertrees [5]. The additional ingredient here is the explicit polynomial constraint that remains valid during each pairwise combination of observations. Its pairing with the two combination inequalities gives a strict moment saving for every parameter in the two branching families when $\lambda > 0$.

The required inequalities are certified on products of probability simplices by nonnegative homogeneous coefficients, using Taylor lower bounds and interpolation with explicitly certified remainders. The interpolation itself is never assumed to approximate the target accurately. Thus the computation verifies fixed polynomial inequalities on all posterior vectors; the preserved constraint on their laws then supplies the tree argument.

Section 2 gives the posterior operations, and Section 3 states the explicit polynomial inequalities and derives their consequences for posterior laws. Section 4 completes the probabilistic argument. Sections 5 and 6 prove the inequalities through a finite exact-arithmetic calculation.

2 Symmetric posterior experiments

The observations below are experiments about an input that is uniform on [4]. A tree combines such experiments in two ways: an observation passes through one broadcast edge, and observations from different branches are combined. We record the exact posterior laws for these operations, keeping separate the independent laws used in a calculation and the actual law of a combined observation. The normalized posterior update and its change of observation law are standard in broadcasting recursions; see [9, Sections 2.2–2.4].

Write

$$\Delta_4 = \left\{ p \in [0, 1]^4 : \sum_{i=1}^4 p_i = 1 \right\}, \quad \mathbf{u} = (1/4, 1/4, 1/4, 1/4),$$

and let $\mathfrak{S}_4$ be the group of permutations of [4]. An experiment consists of a uniform input $I \in [4]$ and an observation Y . Write κ_i for the conditional law of Y given $I = i$, and $\kappa = \frac{1}{4} \sum_i \kappa_i$ for its marginal law. Since each κ_i is absolutely continuous with respect to κ , its posterior vector $p(Y) \in \Delta_4$ satisfies

$$p_i(y) = \frac{1}{4} \frac{d\kappa_i}{d\kappa}(y), \quad \frac{d\kappa_i}{d\kappa}(y) = 4p_i(y). \quad (2.1)$$

These identities also define the posterior on a general observation space, up to a κ -null set. Denote by μ the law of $p(Y)$ under κ . In particular,

$$\int_{\Delta_4} p_i d\mu(p) = \frac{1}{4} \quad (i \in [4]).$$

Conversely, any probability law μ with this barycenter is realized by taking the observation space to be Δ_4 and its conditional laws to be $4p_i d\mu(p)$. We call the experiment symmetric if μ is invariant under all permutations of its four coordinates. This is a condition on the law of the posterior, not on an individual posterior vector.

The following coordinates remove the constant direction:

$$L = \begin{pmatrix} 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}, \quad x = Lp, \quad A(p) = |Lp|^2. \quad (2.2)$$

Since $L^\top L = 4\text{Id} - \mathbf{1}\mathbf{1}^\top$, where $\mathbf{1} = (1, 1, 1, 1)^\top$,

$$A(p) = 4 \sum_{i=1}^4 (p_i - 1/4)^2, \quad 0 \leq A(p) \leq 3, \quad \frac{1}{2} \sum_{i=1}^4 |p_i - 1/4| \leq \frac{1}{2} \sqrt{A(p)}. \quad (2.3)$$

Thus decay of the mean of A will imply nonreconstruction.

Lemma 2.1 (Bayes product). *Let two experiments have the same uniform input and observations that are independent conditional on that input. Let μ and ν be their individual posterior laws. For $p, q \in \Delta_4$, define*

$$U_i(p, q) = 4p_i q_i, \quad z(p, q) = \sum_{i=1}^4 U_i(p, q), \quad P_i(p, q) = \frac{U_i(p, q)}{z(p, q)} \quad \text{when } z(p, q) > 0.$$

Set $P(p, q) = \mathbf{u}$ when $z(p, q) = 0$. The posterior law of the combined experiment, denoted by $\mu \star \nu$, is characterized by

$$\int_{\Delta_4} f(r) d(\mu \star \nu)(r) = \int_{\Delta_4 \times \Delta_4} z(p, q) f(P(p, q)) d\mu(p) d\nu(q) \quad (2.4)$$

for every bounded measurable f . The measure on the right has total mass one. If both experiments are symmetric, their combined experiment is symmetric.

Proof. Let κ, η be the marginal observation laws, with posterior vectors $p(y), q(w)$. Conditional independence and (2.1) show that the joint observation law has density

$$\frac{1}{4} \sum_{i=1}^4 (4p_i(y))(4q_i(w)) = z(p(y), q(w))$$

with respect to $\kappa \otimes \eta$. Bayes' formula then gives the combined posterior P . Pushing this identity forward to the two posterior vectors proves (2.4). Its normalization can also be checked directly:

$$\int z(p, q) d\mu(p) d\nu(q) = 4 \sum_{i=1}^4 \left(\int p_i d\mu \right) \left(\int q_i d\nu \right) = 1.$$

Finally, simultaneous coordinate permutation leaves z unchanged and permutes P by the same permutation. Invariance of μ and ν therefore implies invariance of $\mu \star \nu$. $\square$

In (2.4), p and q are independent under $\mu \otimes \nu$. They generally are not independent under the actual observation law $z d\mu d\nu$. All factorizations of expectations below use the former measure, before the factor z is applied. The choice of P at $z = 0$ never affects a combined expectation.

Lemma 2.2 (Passage through one edge). *Let I be uniform on $[4]$, let J be obtained from I through the channel P_λ , and let Y be an experiment about J , independent of I conditional on J . If μ is the posterior law for J given Y , then the posterior law for I given Y is the pushforward of μ by*

$$C_\lambda(p) = \mathbf{u} + \lambda(p - \mathbf{u}), \quad LC_\lambda(p) = \lambda Lp, \quad A(C_\lambda(p)) = \lambda^2 A(p). \quad (2.5)$$

In particular, this operation preserves symmetry.

Proof. The channel is symmetric and doubly stochastic, so J is uniform and $\Pr(I = i \mid J = j) = P_\lambda(i \mid j)$. If p is the posterior of J given Y , the posterior of I is consequently

$$\sum_{j=1}^4 P_\lambda(i \mid j) p_j = \lambda p_i + \frac{1 - \lambda}{4}.$$

The marginal law of Y is the same whether it is regarded as an experiment about I or about J . This proves the pushforward statement. The remaining identities follow from $L\mathbf{u} = 0$, and symmetry follows because C_λ commutes with coordinate permutations. $\square$

We will also use observed mixtures of experiments. Suppose an index S is independent of the input and, conditional on $S = s$, the observation has posterior law μ_s . If S itself is observed, the resulting posterior law is $\int \mu_s d\Pr_S(s)$. Hence symmetry and any linear inequality on posterior laws are preserved under such mixtures. In the tree application, S will be the offspring count. The uninformative experiment has posterior law concentrated at $\mathbf{u}$; the experiment that reveals the input has posterior law assigning mass $1/4$ to each of the four coordinate vectors.

3 A preserved constraint on posterior laws

The second moment $\mathbb{E}A(p)$ measures information about the input, but the estimate we need for products also involves one higher-order statistic. We now give this statistic explicitly. Its expectation will remain nonnegative throughout the recursion, allowing a strict improvement on second-moment subadditivity.

For $x = Lp$, with L as in Section 2, set

$$B(p) = x_1 x_2 x_3, \quad C(p) = x_1^2 x_2^2 + x_1^2 x_3^2 + x_2^2 x_3^2.$$

Together with $A(p) = x_1^2 + x_2^2 + x_3^2$, these polynomials are invariant under permuting the four entries of p . Indeed, such a permutation acts on x by a permutation of its coordinates and an even number of sign changes. Define H, F, G by the following coefficient table, whose entries are divided by 1000:

	1	A	B	A^2	C	AB	A^3	AC	B^2
H	0	0	1000	-100	-150	300	-20	70	-510
F	0	4000	-3000	700	-4000	-800	0	0	0
G	1000	-1610	-3100	773	4000	-3980	0	0	0

(3.1)

Thus, for example, $H = B - A^2/10 - 3C/20 + 3AB/10 - A^3/50 + 7AC/100 - 51B^2/100$. We also write $H(x)$ when its three coordinate arguments, rather than the probability vector, are supplied.

For $p, q \in \Delta_4$ and $\pi \in \mathfrak{S}_4$, put

$$U_i^\pi = 4p_i q_{\pi(i)}, \quad z_\pi = \sum_{i=1}^4 U_i^\pi, \quad P^\pi = U^\pi / z_\pi \quad (z_\pi > 0).$$

For a continuous function ϕ on Δ_4 , use the notation

$$[z\phi(P)](p, q) = \frac{1}{24} \sum_{\pi \in \mathfrak{S}_4} z_\pi \phi(P^\pi), \quad (3.2)$$

with each summand set to zero when $z_\pi = 0$. Finally, define

$$D_0(p, q) = \frac{A(p)A(q)(A(p) + A(q))}{1000}. \quad (3.3)$$

Proposition 3.1 (Polynomial inequalities). *For every $p, q \in \Delta_4$, the polynomials in (3.1) satisfy $F(p) \geq 0$, $G(p) \geq 0$, and $H(p) \geq -1/2$. For $x = Lp$ and $0 \leq t \leq 1$,*

$$H(tx) \geq t^3 H(x). \quad (3.4)$$

Moreover,

$$[zA(P)](p, q) \leq A(p) + A(q) - H(p)F(q) - F(p)H(q) - D_0(p, q), \quad (3.5)$$

$$[zH(P)](p, q) \geq H(p)G(q) + G(p)H(q). \quad (3.6)$$

Sections 5 and 6 prove this proposition using exact polynomial sign certificates. We first show how it controls posterior laws; this identifies the purpose of each inequality before the coefficient calculation begins.

Let $\mathcal{C}$ be the class of symmetric posterior laws μ for which

$$\mathbb{E}_\mu H(p) \geq 0. \quad (3.7)$$

The condition is linear in the law. It does not require $H(p) \geq 0$ at every posterior: for example, $p = (1/2, 1/2, 0, 0)$ has $H(p) = -3/25$. The completely informative experiment and the uninformative experiment belong to $\mathcal{C}$, because

$$H(e_i) = \frac{13}{100} \quad (1 \leq i \leq 4), \quad H(\mathbf{u}) = 0. \quad (3.8)$$

Here the informative experiment has posterior law uniform on the four point masses e_i .

Proposition 3.2 (Closure and moment saving). *The class $\mathcal{C}$ is preserved by a channel step with $\lambda \in [0, 1]$, by conditionally independent products of experiments, and by mixtures whose mixing label is observed and independent of the input. If $\mu, \nu \in \mathcal{C}$ are the marginal posterior laws of two experiments, then their product posterior law $\mu \star \nu$ satisfies*

$$\begin{aligned} \mathbb{E}_{\mu \star \nu} A &\leq \mathbb{E}_\mu A + \mathbb{E}_\nu A \\ &\quad - \frac{\mathbb{E}_\mu A^2 \mathbb{E}_\nu A + \mathbb{E}_\mu A \mathbb{E}_\nu A^2}{1000}. \end{aligned} \quad (3.9)$$

In particular, when $\mathbb{E}_\mu A = \mathbb{E}_\nu A = b$, the subtracted term is at least $2b^3/1000$.

Proof. By Lemma 2.2, a channel step scales $x = Lp$ by λ without changing the marginal law of the observation. Consequently (3.4) gives

$$\mathbb{E} H(\lambda x) \geq \lambda^3 \mathbb{E} H(x) \geq 0.$$

The resulting law remains symmetric.

For a product, Lemma 2.1 expresses its posterior expectations as $\mathbb{E}_{\mu \otimes \nu}[z\phi(P)]$. The expectation here is over independent marginal posterior laws, before multiplication by the density z . Since ν is symmetric, it is unchanged by permuting its four coordinates. We may therefore replace $z\phi(P)$ by its permutation average whenever ϕ is A or H . Integrating (3.6) and using independence gives

$$\mathbb{E}_{\mu \star \nu} H \geq \mathbb{E}_{\mu} H \mathbb{E}_{\nu} G + \mathbb{E}_{\mu} G \mathbb{E}_{\nu} H \geq 0.$$

Here $G \geq 0$ pointwise, whereas the two expectations of H are nonnegative by membership in $\mathcal{C}$. Symmetry is preserved by Lemma 2.1.

Similarly, integration of (3.5) yields

$$\mathbb{E}_{\mu \star \nu} A \leq \mathbb{E}_{\mu} A + \mathbb{E}_{\nu} A - \mathbb{E}_{\mu} H \mathbb{E}_{\nu} F - \mathbb{E}_{\mu} F \mathbb{E}_{\nu} H - \mathbb{E}_{\mu \otimes \nu} D_0(p, q).$$

The two terms involving F can be discarded because $F \geq 0$ and $\mu, \nu \in \mathcal{C}$. Expanding (3.3) gives (3.9). Since $A \geq 0$, Jensen's inequality gives $\mathbb{E}_{\mu} A^2 \geq (\mathbb{E}_{\mu} A)^2$ and the analogous bound for ν , which proves the last assertion.

Finally, conditioning on an observed mixing label independent of the input leaves the uniform prior unchanged. The posterior law of the mixed experiment is the corresponding mixture of posterior laws. Symmetry and (3.7) are both preserved by this mixture. $\square$

4 Decay on the tree

We now apply Proposition 3.2 to the observations in successive generations. The saving from just two branches will suffice; the remaining branches need only satisfy subadditivity.

For the calculation, supply an ordering of the children at every vertex. In the Galton–Watson model, one may construct this ordered tree by giving each vertex an independent offspring count and numbering its children from 1 to that count. Erasing the numbering gives the unlabelled tree in the theorem. Write $\hat{T}$ for the ordered tree and $\hat{T}_{\leq n}$ for its truncation at depth n . Define

$$Y_n = (\hat{T}_{\leq n}, (\sigma_v)_{v \in L_n}), \quad p_i^{(n)} = \Pr(\sigma_\rho = i \mid Y_n).$$

Every finite-depth truncation is finite almost surely. Let μ_n be the law of $p^{(n)}$, and put

$$m_n = \int_{\Delta_4} A(p) d\mu_n(p). \quad (4.1)$$

In the random-tree case this integral averages over the tree as well as the spins, without conditioning on survival.

There are two information comparisons to check. First, conditional on $\hat{T}_{\leq n}$, the part of $\hat{T}$ below depth n is independent of the root and all spins used in Y_n . This follows from the independence of the tree construction and the channel randomness. Consequently, supplying the entire ordered tree leaves the posterior unchanged:

$$\Pr(\sigma_\rho = i \mid \hat{T}, (\sigma_v)_{v \in L_n}) = p_i^{(n)} \quad \text{almost surely.} \quad (4.2)$$

Second, forgetting the ordering cannot increase expected total variation from the prior. Indeed, for observation sigma-fields $\mathcal{F} \subseteq \mathcal{G}$, the posterior given $\mathcal{F}$ is the conditional expectation of the posterior given $\mathcal{G}$; convexity of the total-variation norm gives the assertion. Combining this observation with (2.3) and Jensen's inequality, the advantage defined in Theorem 1.1 satisfies

$$a_n(d, \lambda) \leq \frac{1}{2} \mathbb{E} \sqrt{A(p^{(n)})} \leq \frac{1}{2} \sqrt{m_n}. \quad (4.3)$$

It therefore remains to prove that m_n tends to zero.

Proposition 4.1 (Moment recursion). *In either tree family of Theorem 1.1, the posterior law μ_n is symmetric and satisfies $\int H d\mu_n \geq 0$ for every $n \geq 0$. Let D denote the offspring count, so $D = d$ on the regular tree and D has law $\text{Poisson}(d)$ on the Galton–Watson tree. Then*

$$m_{n+1} \leq d\lambda^2 m_n - \frac{2}{1000} \Pr(D \geq 2)(\lambda^2 m_n)^3. \quad (4.4)$$

Proof. At depth zero the root spin is revealed. Thus μ_0 assigns mass $1/4$ to each coordinate vector, is symmetric, and satisfies $\int H d\mu_0 = 13/100 > 0$.

Suppose $\mu_n \in \mathcal{C}$. Conditional on $D = k$, each child contributes an experiment obtained by passing a depth- n experiment through one broadcast edge. The branches are independent conditional on the root spin, and their individual posterior laws are all the pushforward $(C_\lambda)_* \mu_n$. For the Galton–Watson tree this uses the independence of the child subtrees from each other and from D ; for the regular tree the subtrees are deterministic copies of the same rooted tree. By Lemma 2.2 and Proposition 3.2, each branch law belongs to $\mathcal{C}$ and has mean A equal to

$$b = \lambda^2 m_n.$$

Combining the branches successively by Lemma 2.1 and Proposition 3.2 preserves membership in $\mathcal{C}$. If $k = 0$, the posterior is $\mathbf{u}$ and has $H = A = 0$. Since D is independent of the root and is observed, mixing these laws over D also preserves membership in $\mathcal{C}$. This proves the induction assertion for μ_{n+1} .

For $k \geq 2$, combine the first two branches before adding the others. Their individual posterior laws lie in $\mathcal{C}$ and both have mean A equal to b . The final assertion of Proposition 3.2 therefore bounds the mean of A after their combination by $2b - 2b^3/1000$. Each additional branch increases this upper bound by at most b , since the combined experiment still belongs to $\mathcal{C}$. For $k = 0$ the mean is zero, and for $k = 1$ it is b . In all cases,

$$\mathbb{E}[A(p^{(n+1)}) \mid D = k] \leq kb - \frac{2b^3}{1000} \mathbf{1}_{\{k \geq 2\}}.$$

Taking expectation over D , whose mean is d , proves (4.4). $\square$

Proof of Theorem 1.1. If $\lambda = 0$, each branch observation is independent of the root; equivalently, (4.4) gives $m_n = 0$ for every $n \geq 1$. Assume henceforth that $\lambda > 0$ and $d\lambda^2 \leq 1$.

On the regular tree, $d \geq 2$ and hence $\Pr(D \geq 2) = 1$. The recursion gives

$$0 \leq m_{n+1} \leq m_n - \frac{2\lambda^6}{1000} m_n^3.$$

Thus (m_n) is decreasing and has a limit $\ell \geq 0$. Passing to the limit yields $\ell \leq \ell - 2\lambda^6 \ell^3/1000$, so $\ell = 0$.

For Poisson offspring,

$$\Pr(D \geq 2) = 1 - (1 + d)e^{-d} > 0 \quad (d > 0).$$

The same argument applies with the positive coefficient $2\lambda^6(1 - (1 + d)e^{-d})/1000$ in place of $2\lambda^6/1000$. In particular, it covers $d \leq 1$, including the noiseless critical case $d = \lambda = 1$, directly in the unconditioned tree law. No step requires an infinite tree or a nonempty observed generation: when L_n is empty, the tree observation is independent of the root and the posterior is $\mathbf{u}$.

Both arguments allow $d\lambda^2 = 1$; the strictly positive cubic term is what forces decay at that boundary. Finally, (4.3) gives $a_n(d, \lambda) \rightarrow 0$ in each family. $\square$

5 Exact polynomial certificates

We now prove the polynomial inequalities in Proposition 3.1. The single-posterior inequalities follow from small homogeneous coefficient calculations. For the two-posterior inequalities, we give two sufficient tests on a product of simplices. Section 6 applies those tests to a finite subdivision and completes the proof.

5.1 Ordered probabilities and homogeneous coefficients

Let

$$\mathcal{S} = \{p \in \Delta_4 : p_1 \geq p_2 \geq p_3 \geq p_4\}, \quad S_i = \frac{1}{i+1} \sum_{v=1}^{i+1} e_v \quad (0 \leq i \leq 3),$$

where e_v is the v th standard basis vector of $\mathbb{R}^4$. The set $\mathcal{S}$ is the simplex with vertices S_0, S_1, S_2, S_3 . Indeed, on setting $p_5 = 0$, its barycentric coordinates are

$$X_i = (i+1)(p_{i+1} - p_{i+2}), \quad p = \sum_{i=0}^3 X_i S_i, \quad \sum_{i=0}^3 X_i = 1. \quad (5.1)$$

All X_i are nonnegative. Conversely, each such convex combination lies in $\mathcal{S}$.

A permutation of the four probabilities acts on $x = Lp$ by permuting its three coordinates and changing an even number of their signs. One way to check this is to note that, on the hyperplane $\sum_v p_v = 1$, $p = u + L^\top x/4$. For a permutation matrix Π , the induced matrix is $L\Pi L^\top/4$; it is a signed permutation matrix whose three nonzero entries have product 1. Consequently A, B, C , and hence H, F, G , are invariant. It is therefore enough to check the single-posterior inequalities on $\mathcal{S}$. The permutation-averaged two-posterior expressions are invariant under separate permutations of p and q : a permutation of q reindexes the average, and a permutation of p can be moved to q by simultaneous permutation of the output coordinates. We may thus restrict both inputs to $\mathcal{S}$.

We use ordinary monomial coefficients throughout. For $r \geq 0$, put

$$\mathcal{I}_r = \{I \in \mathbb{Z}_{\geq 0}^4 : |I| = r\}, \quad X^I = \prod_{i=0}^3 X_i^{I_i}, \quad W_r(X) = \left(\sum_{i=0}^3 X_i \right)^r.$$

The coefficient of X^I in W_r is the multinomial coefficient $\binom{r}{I} = r! / \prod_i I_i!$. A polynomial of degree at most r can be homogenized to degree r by multiplying each degree- s monomial by W_{r-s} . This does not change its values when $\sum_i X_i = 1$. If all coefficients of a homogeneous polynomial are nonnegative, then the polynomial is nonnegative on the simplex. No converse is required. The coefficients in the simplicial Bernstein basis differ from these ordinary homogeneous coefficients by positive multinomial factors, so their signs agree. This is the coefficient-positivity principle behind simplicial Bernstein bounds and their refinement by subdivision; see [7].

The calculations below use only coefficient addition and the product rule

$$[X^I](RS) = \sum_{J+K=I} [X^J]R[X^K]S. \quad (5.2)$$

For two sets of variables, write $W_r(X, Y) = W_r(X)W_r(Y)$. A bihomogeneous polynomial of bidegree (r, r) , meaning homogeneous of degree r in each set of variables, is stored as the matrix of its coefficients $[X^I Y^J]$, with $I, J \in \mathcal{I}_r$. Multiplication is convolution in both indices; multiplication by $W_s(X, Y)$ raises both homogeneous degrees by s . A monomial of degree s in X and degree t in Y is homogenized to bidegree (r, r) by multiplying it by $W_{r-s}(X)W_{r-t}(Y)$.

Polynomial R	Degree r	Number of coefficients	Minimum
$H + 1/2$	6	84	19/50
F	5	56	0
G	14	680	87/1000
$(3 - \mathcal{E})H$	6	84	0

Table 1: Minimum ordinary monomial coefficients of the homogeneous forms (5.4), computed exactly over $\mathbb{Q}$.

5.2 The single-posterior checks

The coordinate forms on $\mathcal{S}$ are particularly small:

$$L \sum_{i=0}^3 X_i S_i = \left(X_0 + X_1 + \frac{X_2}{3}, X_0 + \frac{X_2}{3}, X_0 - \frac{X_2}{3} \right). \quad (5.3)$$

Let $\Phi_0, \dots, \Phi_8$ be the polynomials $1, A, B, A^2, C, AB, A^3, AC, B^2$ in the three coordinates, and let

$$(e_0, \dots, e_8) = (0, 2, 3, 4, 4, 5, 6, 6, 6)$$

be their degrees. If $R = \sum_j c_j \Phi_j$, its degree- r homogeneous form on $\mathcal{S}$ is

$$\hat{R}_r(X) = \sum_{j: c_j \neq 0} c_j \Phi_j \left(L \sum_{i=0}^3 X_i S_i \right) W_{r-e_j}(X), \quad r \geq \max_{c_j \neq 0} e_j. \quad (5.4)$$

Together with (5.3) and (5.2), this formula specifies every coefficient in the following check.

Lemma 5.1 (Single-posterior certificate). *For every $p \in \Delta_4$,*

$$H(p) \geq -\frac{1}{2}, \quad F(p) \geq 0, \quad G(p) \geq 0.$$

Moreover, if $x = Lp$ and $0 \leq \ell \leq 1$, then $H(\ell x) \geq \ell^3 H(x)$, where H in this formula is regarded as a polynomial in the three coordinates.

Proof. Define the Euler operator on those coordinates by $\mathcal{E} = \sum_{j=1}^3 x_j \partial_{x_j}$. Thus $\mathcal{E}\Phi_j = e_j \Phi_j$. The exact coefficient minima in Table 1 are obtained from (5.4). The first row adds $1/2$ to the constant coefficient of H ; the last row replaces each coefficient c_j of H by $(3 - e_j)c_j$. The full scalar verifier is included in Appendix A.

Coefficient nonnegativity proves the first three assertions and $(3 - \mathcal{E})H \geq 0$ on LS . Permutation invariance extends these statements to $L\Delta_4$. For $x \in L\Delta_4$ and $0 < \ell \leq 1$, the point ℓx also lies in $L\Delta_4$, since it corresponds to $\ell p + (1 - \ell)\mathbf{u}$. Therefore

$$\frac{d}{d\ell}(\ell^{-3}H(\ell x)) = \ell^{-4}(\mathcal{E} - 3)H(\ell x) \leq 0.$$

Comparing ℓ with 1 gives the asserted scaling inequality. At $\ell = 0$ it follows from $H(0) = 0$. $\square$

5.3 The two rational inequalities

It remains to prove the two-posterior inequalities. We express each as the nonnegativity of a rational function on $\mathcal{S} \times \mathcal{S}$. For $\pi \in \mathfrak{S}_4$, let

$$U_v^\pi = 4p_v q_{\pi(v)}, \quad z_\pi = \sum_{v=1}^4 U_v^\pi, \quad P^\pi = U^\pi / z_\pi \quad (z_\pi > 0).$$

Set

$$\begin{aligned} K_a &= 24(A(p) + A(q) - H(p)F(q) - F(p)H(q) - D_0(p, q)), \\ K_b &= -24(H(p)G(q) + G(p)H(q)). \end{aligned} \tag{5.5}$$

The remaining data are

row	k	N_π	m
a	1	$-\sum_{j=1}^3 (LU^\pi)_j^2$	3
b	5	$z_\pi^6 H(U^\pi / z_\pi)$	1/2

(5.6)

In row b , the expression for N_π is a polynomial: each degree- s term of H becomes the corresponding term in LU^π multiplied by z_π^{6-s} . For either row define

$$\mathcal{R} = K + \sum_{\pi \in \mathfrak{S}_4} \frac{N_\pi}{z_\pi^k}. \tag{5.7}$$

In row a , the summand is $-z_\pi A(P^\pi)$; in row b , it is $z_\pi H(P^\pi)$. Thus $\mathcal{R} \geq 0$ is exactly the corresponding inequality of Proposition 3.1, multiplied by 24.

These rational functions have continuous extensions to their apparent singularities. All coordinates of U^π are nonnegative, so $z_\pi = 0$ implies $U^\pi = 0$. Since $P^\pi \in \Delta_4$ whenever $z_\pi > 0$, both $A(P^\pi)$ and $H(P^\pi)$ are bounded independently of p, q . Hence the corresponding weighted summand tends to 0 as $z_\pi \rightarrow 0$. We assign it that value on the boundary.

Now take any two nondegenerate subsimplices of $\mathcal{S}$, with vertex lists $a = (a_0, \dots, a_3)$ and $b = (b_0, \dots, b_3)$, and write

$$p = \sum_{i=0}^3 X_i a_i, \quad q = \sum_{j=0}^3 Y_j b_j, \quad X_i, Y_j \geq 0, \quad \sum_i X_i = \sum_j Y_j = 1.$$

The ordinary bidegree- $(1, 1)$ coefficient of U_v^π at $X_i Y_j$ is $4a_{iv} b_{j, \pi(v)}$. In particular, the coefficients of z_π are nonnegative. Both p and q are strictly positive on the relative interior of their subsimplices: a full-dimensional simplex cannot lie in a coordinate face of Δ_4 . Thus every z_π is strictly positive there. We represent K homogeneously with bidegree $(6, 6)$ and N_π with bidegree $(k+1, k+1)$.

The next two tests certify $\mathcal{R} \geq 0$ on such a product of simplices. They establish lower bounds on the entire region, not just at the points used to construct the polynomials.

5.4 A Taylor lower bound: Type T

For a positive integer k , define

$$t_k(z) = \sum_{i=0}^5 \binom{-k}{i} (z-1)^i.$$

For every $z > 0$, Taylor's theorem gives

$$z^{-k} - t_k(z) = \frac{k(k+1) \cdots (k+5)}{6!} \xi^{-k-6} (z-1)^6 \geq 0 \quad (5.8)$$

for some ξ between 1 and z ; at $z = 1$ the identity is immediate. The bound holds for the full positive half-line, with no restriction such as $|z - 1| < 1$.

For row a , $A(P) \leq 3$ on Δ_4 , because each coordinate of LP lies in $[-1, 1]$. For row b , Lemma 5.1 gives $H(P) \geq -1/2$. Consequently, in either row,

$$N_\pi + mz_\pi^{k+1} \geq 0. \quad (5.9)$$

Lemma 5.2 (Type T certificate). *For either row of (5.6), form the polynomial*

$$T = K + \sum_{\pi \in \mathfrak{S}_4} ((N_\pi + mz_\pi^{k+1})t_k(z_\pi) - mz_\pi). \quad (5.10)$$

If its homogeneous coefficient matrix in bidegree $(k+6, k+6)$ has no negative entries, then $\mathcal{R} \geq 0$ on the product of simplices.

Proof. On the relative interior, multiply (5.8) by the nonnegative factor in (5.9) and subtract mz_π . This yields

$$\frac{N_\pi}{z_\pi^k} \geq (N_\pi + mz_\pi^{k+1})t_k(z_\pi) - mz_\pi.$$

Summing gives $\mathcal{R} \geq T$. The coefficient hypothesis makes T nonnegative on the product simplex. The continuous extension of $\mathcal{R}$ then proves the boundary cases as well. $\square$

5.5 A polynomial lower bound with a certified remainder: Type P

The second test constructs a lower bound for each rational summand separately. Start with any bihomogeneous polynomial J_π of bidegree $(3, 3)$. The way it is chosen affects the strength of the test, but not its validity. We seek a constant $c_\pi \geq 0$ such that

$$\frac{N_\pi}{z_\pi^k} \geq J_\pi - c_\pi$$

throughout the product simplex. On the relative interior, clearing the positive denominator reduces this to nonnegativity of $N_\pi - z_\pi^k J_\pi + c_\pi z_\pi^k$. To test that remainder by homogeneous coefficients, form in bidegree $(k+3, k+3)$ the arrays

$$O_\pi = W_2 N_\pi - z_\pi^k J_\pi, \quad Q_\pi = z_\pi^k W_3, \quad (5.11)$$

where the arguments (X, Y) of W_2, W_3 are suppressed. Every coefficient of Q_π is nonnegative.

Fix $M = 2^{32}$. If an entry $(O_\pi)_{IJ}$ is negative and the corresponding entry $(Q_\pi)_{IJ}$ is zero, the test is not applicable. Otherwise define

$$c_\pi = \frac{1}{M} \max \left(\{0\} \cup \left\{ \left\lceil \frac{-M(O_\pi)_{IJ}}{(Q_\pi)_{IJ}} \right\rceil : (O_\pi)_{IJ} < 0 \right\} \right). \quad (5.12)$$

By construction, $O_\pi + c_\pi Q_\pi$ is coefficientwise nonnegative.

Lemma 5.3 (Type P certificate). *Suppose (5.12) is defined for every $\pi \in \mathfrak{S}_4$. If*

$$K + \sum_{\pi \in \mathfrak{S}_4} (J_\pi - c_\pi W_3) \quad (5.13)$$

has nonnegative homogeneous coefficients in bidegree (6, 6), then $\mathcal{R} \geq 0$ on the product of simplices.

Proof. On the product simplex, $W_2 = W_3 = 1$. On its relative interior, (5.11) gives the exact identity

$$\frac{N_\pi}{z_\pi^k} = J_\pi - c_\pi + \frac{O_\pi + c_\pi Q_\pi}{z_\pi^k} \geq J_\pi - c_\pi.$$

Summing and using the coefficient hypothesis proves the claim on the relative interior. Continuity gives the boundary cases. $\square$

For reproducibility, we specify the polynomials J_π used in the finite check. For $I \in \mathcal{I}_3$, define the cardinal polynomial

$$\ell_I(X) = \prod_{i=0}^3 \binom{3X_i}{I_i}, \quad \binom{t}{j} = \frac{t(t-1) \cdots (t-j+1)}{j!}, \quad \binom{t}{0} = 1.$$

At a lattice point $J/3$, with $J \in \mathcal{I}_3$, its value is $\prod_i \binom{J_i}{I_i} = \mathbf{1}_{\{I=J\}}$. Indeed, if $I \neq J$ then some $J_i < I_i$, since $|I| = |J| = 3$. These 20 cardinal polynomials therefore give interpolation of degree at most three on the three-dimensional simplex. In coefficient arrays we use their homogeneous degree-three forms, obtained by replacing each factor $3X_i - h$ by $3X_i - h \sum_j X_j$.

Let $\text{trunc}_M(t)$ denote truncation toward zero to a multiple of $1/M$. Evaluate the continuous rational summand N_π/z_π^k at the 20^2 node pairs $(I/3, J/3)$, assigning zero when $z_\pi = 0$, and put

$$J_\pi(X, Y) = \sum_{I, J \in \mathcal{I}_3} \text{trunc}_M \left(\frac{N_\pi(I/3, J/3)}{z_\pi(I/3, J/3)^k} \right) \ell_I(X) \ell_J(Y). \quad (5.14)$$

The convention at vanishing denominators is understood in this formula. For the rational vertex lists used in Section 6, all these node values are rational. Their evaluation, truncation, and interpolation can therefore be performed in exact rational arithmetic. Lemma 5.3 does not estimate interpolation error or truncation error: it certifies the remainder of the actual rational polynomial J_π chosen by (5.14). Thus the verification remains exact despite the finite interpolation grid and truncation.

The single-posterior inequalities are now proved. For the two remaining inequalities, the proof has been reduced to finding a finite cover of $\mathcal{S} \times \mathcal{S}$ by products of simplices on each of which either Lemma 5.2 or Lemma 5.3 applies. We carry out that finite verification next.

6 Finite verification

The certificates in Section 5 reduce the two product inequalities to signs of rational coefficient arrays. We now specify a finite subdivision of $\mathcal{S} \times \mathcal{S}$ on which those signs can be checked. The subdivision and all coefficient operations are exact. The complete verifier is printed in Appendix A.

6.1 Subdivision and coverage

A cell is specified by two affinely independent lists of four rational vertices $a = (a_0, a_1, a_2, a_3)$ and $b = (b_0, b_1, b_2, b_3)$ in $\mathcal{S}$; its domain is $\text{conv}(a) \times \text{conv}(b)$. Initially both lists are (S_0, S_1, S_2, S_3) . On a cell we substitute

$$p = \sum_{i=0}^3 X_i a_i, \quad q = \sum_{j=0}^3 Y_j b_j, \quad X, Y \in \Delta_4,$$

into the arrays of Section 5.

Lemma 6.1 (Midpoint subdivision). *Let $\varepsilon_0, \dots, \varepsilon_3$ be the unit vectors in the barycentric coordinate space. Replace vertex a_i by $(a_i + a_j)/2$, where $i \neq j$, leaving the other vertices unchanged. For an ordinary homogeneous coefficient indexed by $I \in \mathcal{I}_r$, its contributions to the new coefficients are indexed by*

$$I + t\varepsilon_i - t\varepsilon_j, \quad 0 \leq t \leq I_j, \quad \text{with multiplier } 2^{-I_i-t} \binom{I_j}{t}. \quad (6.1)$$

The same formula holds in the Y index when a vertex of b is replaced. The two cells obtained by replacing vertex i , respectively vertex j , by their midpoint cover the original cell.

Proof. Write X' for the new barycentric coordinates. The old coordinates satisfy $X_i = X'_i/2$, $X_j = X'_j + X'_i/2$, with the others unchanged. Expanding $X_i^{I_i} X_j^{I_j}$ gives (6.1). The first child contains precisely the barycentric points with $X_i \leq X_j$: one can take $X'_i = 2X_i$ and $X'_j = X_j - X_i$. The second child contains the points with $X_j \leq X_i$. These two inequalities exhaust the original simplex. Each replacement preserves affine independence, since its barycentric change of coordinates has nonzero determinant. $\square$

Here is the entire decision rule for one row of (5.6). Start at depth zero. At each cell:

1. Try the Type T coefficient test of Lemma 5.2. Accept the cell if every coefficient is nonnegative.
2. If neither vertex list contains $\mathbf{u}$, try the Type P test of Lemma 5.3, with $M = 2^{32}$. If the correction (5.12) is undefined, report failure immediately. Otherwise accept the cell if its coefficient test succeeds.
3. If neither test has accepted the cell and its depth is 24, report failure. Otherwise select an edge as follows. For $y \in \{0, 1\}$, put $c = a$ when $y = 0$ and $c = b$ when $y = 1$. Among all $i > j$, maximize lexicographically the tuple

$$(w, y, i, j), \quad w = \|c_i - c_j\|_2^2 \begin{cases} 16, & c_i = \mathbf{u} \text{ or } c_j = \mathbf{u}, \\ 1, & \text{otherwise.} \end{cases}$$

Replace vertex j by the midpoint and recursively check that child; then replace vertex i by the midpoint and check the other child. Both children have depth one greater than their parent.

Skipping Type P on cells having a uniform vertex is part of this specified rule; no claim about the success or failure of that test on such cells is needed. Lemma 6.1 shows that successful termination proves the required sign on the entire initial domain.

6.2 The completed verification

The verifier represents every coefficient matrix by integer numerators and a common positive denominator, using arbitrary-precision integers for both. Its polynomial arithmetic consists of convolution, degree elevation, rational addition, and the exact substitutions of Lemma 6.1. The fixed-width quantities used for indices, counters, vertices, and small combinatorial factors remain within their integer ranges. Appendix A proves these bounds and identifies the implementation routines with the mathematical certificates. Thus a successful run verifies exact coefficient signs, without a numerical tolerance.

Proposition 6.2 (Finite sign verification). *For each row of (5.6), the decision rule above terminates without failure, with the counts in Table 2. Every terminal cell is accepted by Type T or Type P. Consequently both rational expressions (5.7) are nonnegative on $\mathcal{S} \times \mathcal{S}$.*

Row	Cells visited	Maximum depth	Type T leaves	Type P leaves	Unresolved
(a)	4479	19	166	2074	0
(b)	1935	20	97	871	0

Table 2: Exact subdivision counts. The initial cell has depth zero; visited cells include both internal nodes and terminal cells.

Proof. Run the integer verifier in Appendix A, once in each mode, with assertions enabled. It returns the two lines

```
done 4479 19 lp=2074 lt=166
done 1935 20 lp=871 lt=97
```

Here `lp` and `lt` count the terminal cells accepted by Types P and T. The substitutions are those of Lemma 6.1, and Appendix A.1 verifies their exact implementation and the coefficient arithmetic. Lemmas 5.2 and 5.3 certify the sign on each accepted cell, and Lemma 6.1 proves coverage. The continuous extension at zero denominators established in Section 5 includes the cell boundaries. $\square$

For reproduction, the paper directory contains `verification/checks/certificate.cpp` and `verification/checks/small_polynomials.py`, printed in full in Appendix A. The first requires a C++17 compiler with GCC-compatible standard-library headers and the Boost multiprecision headers; the second uses only the Python 3 standard library. For example, from the paper's `verification/` directory run

```
mkdir -p artifacts
g++ -std=c++17 -O3 -UNDEBUG checks/certificate.cpp \
    -o artifacts/certificate
./artifacts/certificate
./artifacts/certificate b
python3 checks/small_polynomials.py
```

The C++ program writes progress and its final result to standard error. Its no-argument mode checks row (a); any argument selects row (b). The Python program independently computes the four scalar coefficient minima of Lemma 5.1, using rational convolution, and checks all 24 coordinate actions. Assertions must remain enabled in both programs: do not define `NDEBUG` or run Python with `-O`. Together with the scalar checks, Proposition 6.2 completes the proof of Proposition 3.1.

References

- [1] N. Bhatnagar and E. Maneva, *A computational method for bounding the probability of reconstruction on trees*, SIAM J. Discrete Math. **25** (2011), no. 2, 854–871. doi:10.1137/090751244.
- [2] P. M. Bleher, J. Ruiz, and V. A. Zagrebnov, *On the purity of the limiting Gibbs state for the Ising model on the Bethe lattice*, J. Stat. Phys. **79** (1995), nos. 1–2, 473–482. doi:10.1007/BF02179399.
- [3] W. Evans, C. Kenyon, Y. Peres, and L. J. Schulman, *Broadcasting on trees and the Ising model*, Ann. Appl. Probab. **10** (2000), no. 2, 410–433. doi:10.1214/aoap/1019487349.
- [4] M. Formentin and C. Külske, *A symmetric entropy bound on the non-reconstruction regime of Markov chains on Galton–Watson trees*, Electron. Commun. Probab. **14** (2009), 587–596. doi:10.1214/ECP.v14-1516.
- [5] Y. Gu, *Exact reconstruction thresholds on hypertrees over a symmetric binary alphabet*, preprint (2026). arXiv:2606.21699.
- [6] H. Kesten and B. P. Stigum, *Additional limit theorems for indecomposable multidimensional Galton–Watson processes*, Ann. Math. Statist. **37** (1966), no. 6, 1463–1481. doi:10.1214/aoms/1177699139.
- [7] R. Leroy, *Convergence under subdivision and complexity of polynomial minimization in the simplicial Bernstein basis*, Reliable Computing **17** (2012), 11–21.
- [8] R. Lyons, *Random walks and percolation on trees*, Ann. Probab. **18** (1990), no. 3, 931–958. doi:10.1214/aop/1176990730.
- [9] M. Mézard and A. Montanari, *Reconstruction on trees and spin glass transition*, J. Stat. Phys. **124** (2006), no. 6, 1317–1350. doi:10.1007/s10955-006-9162-3.
- [10] E. Mossel, *Reconstruction on trees: Beating the second eigenvalue*, Ann. Appl. Probab. **11** (2001), no. 1, 285–300. doi:10.1214/aoap/998926994.
- [11] E. Mossel and Y. Peres, *Information flow on trees*, Ann. Appl. Probab. **13** (2003), no. 3, 817–844. doi:10.1214/aoap/1060202828.
- [12] E. Mossel, A. Sly, and Y. Sohn, *Exact phase transitions for stochastic block models and reconstruction on trees*, Ann. Probab. **53** (2025), no. 3, 967–1018. doi:10.1214/24-AOP1723. Author preprint: arXiv:2212.03362.
- [13] F. Ricci-Tersenghi, G. Semerjian, and L. Zdeborová, *Typology of phase transitions in Bayesian inference problems*, Phys. Rev. E **99** (2019), no. 4, 042109. doi:10.1103/PhysRevE.99.042109.
- [14] A. Sly, *Reconstruction for the Potts model*, Ann. Probab. **39** (2011), no. 4, 1365–1406. doi:10.1214/10-AOP584.

A Exact implementation and complete verifiers

This appendix identifies the array operations with the certificates of Section 5 and bounds the auxiliary fixed-width arithmetic. The complete source listings follow these implementation details. The command lines and expected C++ output are given in Section 6.

A.1 Exact coefficient arithmetic

The C++ type `P` represents a rational coefficient matrix of bidegree (r, r) by an integer array and a common positive denominator. Its entries are indexed by $\mathcal{I}_r$ in descending lexicographic order. Integer numerators and denominators use `boost::multiprecision::cpp_int`. The routine `ones(r)` constructs W_r ; multiplication is ordinary coefficient convolution, and `lift` raises both degrees by multiplying by W_1 . Addition first lifts to a common degree and then uses a common denominator. Thus signs are tested on integer numerators, without a floating-point tolerance.

We explain the correspondence between the remaining routines and the mathematical certificates; this also fixes the normalizations in the listing. Initially the coordinates of every vertex are stored as integers at scale

$$\kappa = 12 \cdot 2^{24}. \quad (\text{A.1})$$

Removing the factor 2^{24} leaves the denominator 12 for the individual coordinates. The coefficients of U_v are $4(a_i)_v(b_j)_{\pi(v)}$; their initialization removes 2^{48} and uses denominator 144. The routine `inv` forms the nine invariant polynomials in their stated order. The routine `dot` forms a linear combination divided by 1000 and homogenizes lower-degree terms with its third argument. With that argument equal to the homogeneous one it gives $H(p)$, $F(p)$, or $G(p)$; with that argument equal to z it gives $z^6 H(U/z)$.

There is one useful economy in constructing K . A polynomial depending only on X is nevertheless stored with both degrees equal, so it has the form $f(X)(\sum_j Y_j)^r$. Its column indexed by $(r, 0, 0, 0)$ contains exactly the coefficients of f . Taking this column and the analogous row of a polynomial depending only on Y gives the outer product used by `tensor`. The factor 24 in `make` then gives exactly the K of (5.5). The array `t` is the Type T polynomial; it is constructed once and thereafter restricted by midpoint substitution. The arrays `N` and `D` store N_π and z_π , respectively, for the 24 permutations in lexicographic order.

The Type P interpolation uses ordinary matrix multiplication. The matrix `ev(r, 3)` has entries I^J , where $I \in \mathcal{I}_3$ indexes a node and $J \in \mathcal{I}_r$ indexes a monomial. Consequently the two-sided evaluation `act` must be divided by 3^{2r} . The listing includes this factor explicitly, using 9 when $r = 1$. The routine `inverse` constructs, rather than numerically inverts, the cardinal polynomials in (5.14). It expands their homogeneous linear factors $3X_h - m \sum_i X_i$, stores their coefficients multiplied by 960, and divides by the factorial denominators. Two-sided interpolation therefore requires division by 960^2 .

After the node values of N_π/z_π^k have been truncated toward zero in multiples of M^{-1} , `testP` constructs the interpolant J_π . Its subtraction `o=b-o` implicitly lifts N_π by W_2 , and hence constructs precisely $O_\pi = W_2 N_\pi - z_\pi^k J_\pi$. The array `w` is $Q_\pi = z_\pi^k W_3$. For each negative coefficient of O_π , the program asserts that the corresponding coefficient of Q_π is positive. A violation aborts the verification rather than merely rejecting that cell; no such violation occurs in the completed runs. It then computes the integer ceiling in (5.12), by positive integer division, and retains the largest value. The final test is exactly the coefficient test of Lemma 5.3.

Finally, `sub` implements (6.1) with 2^r cleared. Its integer multiplier is

$$C_t = 2^{r-I_i-t} \binom{I_j}{t}, \quad C_{t+1} = \frac{C_t(I_j - t)}{2(t+1)}. \quad (\text{A.2})$$

Here $r - I_i - t \geq 0$ for every used term, so the indicated division is exact. The resulting matrix denominator is multiplied by 2^r .

A.2 Bounds for fixed-width arithmetic

Only indices, counters, vertex coordinates, and small combinatorial factors use fixed-width integer types. Take `int` to have at least 32 bits and `long long` to have at least 64 bits. The following bounds cover every such operation in the listing.

All polynomial matrices in the C++ verifier have bidegree at most 11. Thus there are at most $\binom{14}{3} = 364$ indices in one dimension and 132496 entries in a matrix. The depth limit 24 allows at most $2^{25} - 1$ visited cells, so the counters for visited cells and terminal cells also fit a 32-bit signed integer. The largest factorial initialized is $19! = 121\,645\,100\,408\,832\,000 < 2^{63}$, and products of factorials appearing in a multinomial denominator do not exceed this bound. The largest multinomial coefficient through degree 11 is 92400, whose square is $8\,537\,760\,000 < 2^{63}$.

Each scaled vertex coordinate belongs to $[0, \kappa]$. Initial products are bounded by $4\kappa^2$, and even the conservative bound

$$64\kappa^2 = 2\,594\,073\,385\,365\,405\,696 < 2^{63}$$

covers the weighted squared edge distances. The initial coordinates are multiples of 2^{24} , so all midpoint coordinates are integers through depth 24. A point of $\mathcal{S}$ has fourth coordinate at most $1/4$, with equality only at $\mathbf{u}$. Thus the test that a stored fourth coordinate is $\kappa/4$ detects a uniform vertex exactly.

The multipliers in (A.2), including the intermediate numerator of the recurrence, are bounded by $11 \cdot 2^{11} \binom{11}{5} < 2^{24}$. The evaluation matrices contain integers at most 3^6 , and the cardinal-polynomial construction uses only three linear factors starting from 960. Even bounding the sum of absolute coefficients at each multiplication by a factor 8 gives $960 \cdot 8^3 < 2^{19}$. All products with polynomial coefficients are already operations on unlimited integers. These bounds exclude fixed-width overflow; in particular, the exact sign tests do not rely on floating-point arithmetic or machine rounding.

A.3 Rational sign certificates

```

1 #include<bits/stdc++.h>
2 #include<boost/multiprecision/cpp_int.hpp>
3 using namespace std;
4 using R=long long;
5 using RR=boost::multiprecision::cpp_int;
6
7 RR Pow(RR a,int n){RR r=1;while(n-->0)r*=a;return r;}
8 RR gcdB(RR a,RR b){RR c;while(b!=0){c=a%b;a=b;b=c;}return a;}
9 using v=array<int,4>;
10 using V=array<R,4>;
11 vector<v> l[20];int id_[20][20][20][20]; R fact[21];
12 int id(int d,v x){return id_[d][x[0]][x[1]][x[2]];} int S(int d){return l[d].size();}
13 v operator+(v a,v b){for(int i=0;i<4;i++)a[i]+=b[i];return a;}
14 v e(int j){v a={};a[j]=1;return a;}
15 void init(){
16     fact[0]=1;
17     for(int d=0;d<20;d++){ if(d)fact[d]=d*fact[d-1]; // l[1] idx 0..3
18     for(int i=d;i>=0;i--){for(int j=d-i;j>=0;j--){for(int k=d-i-j;k>=0;k--){id_[d][i][j][k]=S(d);l[d].push_back({i,
19         j,k,d-i-j-k});}}
20 }
21 R coef(v a){return fact[accumulate(a.begin(),a.end(),0)]/(fact[a[0]]*fact[a[1]]*fact[a[2]]*fact[a[3]]);}
22 struct P{
23     int d;RR den=1;vector<RR> b;
24     explicit P(int n,R r=0):d(n),b(S(n)*S(n),r){}
25     RR & operator()(int i,int j){return b[i*S(d)+j];}
26     RR const& operator()(int i,int j)const{return b[i*S(d)+j];}
27 };

```

```

27 P ones(int d){P r(d);for(int i=0;i<S(d);i++)for(int j=0;j<S(d);j++) r(i,j)=coef(l[d][i])*coef(l[d][j]); return
    r;}
28 P operator*(const P&a,const P&b){
29     int m=a.d,n=b.d,d=m+n;
30     P c(d); vector<int>w(S(m)*S(n));
31     for(int i=0,h=0;i<S(m);i++)for(int j=0;j<S(n);j++)w[h++]=id(d,l[m][i]+l[n][j]);
32     // b smaller
33     for(int j=0;j<S(n);j++)for(int k=0;k<S(n);k++){RR B=b(j,k);if(B==0)continue;for(int i=0;i<S(m);i++){
34         RR*z=&c(w[i*S(n)+j],0); for(int h=0;h<S(m);h++) z[w[h*S(n)+k]]+=a(i,h)*B;
35     }}
36     c.den=a.den*b.den;return c;
37 }
38 P operator*(RR r,P a){for(auto&x:a.b)x*=r;return a;}
39 P divi(P a,R b){ a.den*=b;return a;}
40 P lift(P a,int d){while(a.d<d)a=a*ones(1);return a;}
41 P operator+(P a,P const&b){if(a.d<b.d)a=lift(a,b.d); if(b.d<a.d)return a+lift(b,a.d);RR g=gcdB(a.den,b.den), r
    =a.den/g,h=b.den/g;for(int i=0;i<a.b.size();i++)a.b[i]=a.b[i]*h+b.b[i]*r;a.den*=h;return a;}
42 P operator-(P a,P const&b){return a+(R(-1)*b);}
43
44 using shape=array<V,4>;// stores scaled vertex coordinates
45 shape start(){shape m={};for(int i=0;i<4;i++)for(int j=0;j<=i;j++)m[i][j]=12*(1<<24)/(i+1);return m;}
46 const int had[3][4]={1,1,-1,-1},{1,-1,1,-1},{1,-1,-1,1};
47 vector<R> h={0,0,1000,-100,-150,300,-20,70,-510};
48 vector<R> f={0,4000,-3000,700,-4000,-800,0,0,0};
49 vector<R> g={1000,-1610,-3100,773,4000,-3980,0,0,0};int deg[]={0,2,3,4,4,5,6,6,6};
50 vector<P> inv(array<P,4>&m){
51     vector<P>x(3,P(1));for(int i=0;i<3;i++)for(int j=0;j<4;j++)x[i]=x[i]+R(had[i][j])*m[j];
52     P u=x[0]*x[1]*x[2];
53     P a=x[0]*x[0],b=x[1]*x[1],c=x[2]*x[2],t=a+b+c,s=t*t,p=a*b+a*c+b*c;
54     return {ones(0),t,u,s,p,u*t,s*t,p*t,u*u};
55 }
56 P dot(vector<R>&c,vector<P>&x,P&z){
57     int i=c.size()-1;while(!c[i])i--;
58     // efficiently
59     P sum=P(0);int d=x[i].d;
60     for(int j=0;j<=i;j++){if(c[j]){
61         P r=x[j];while(r.d<d)r=r*z;sum=sum+c[j]*r;}}
62     return divi(sum,1000);
63 }
64 struct dat{shape a,b; P K; vector<P>N;vector<P>D;P t;
65 int k;
66 };
67 int mode=0;P operator*(P a,R r){return r*a;} // move earlier
68 // subdivide coefficients homogeneous actual
69 P sub(P const&o,bool y,int i,int j){//replace vertex i by midpoint i,j. new poly o(old) old variable w_i=w_i'/2
    and w_j=w_j'+w_i'/2
70     int d=o.d; P r(d);
71     for(int u=0;u<S(d);u++){
72         v q=l[d][u]; R c=R(1)<<(d-q[i]);
73         for(int b=0;b<=l[d][u][j];b++){
74             int H=id(d,q);if(!y){RR*x=&r(H,0);for(int n=0;n<S(d);n++)x[n]+=c*o(u,n);}
75             else for(int n=0;n<S(d);n++) r(n,H)+=c*o(n,u);
76             c=(c*q[j])/(2*(b+1));q[i]++;q[j]--;
77         }}
78     r.den=o.den*(1<<d); return r;
79 }
80 // initialization Taylor lower
81 static dat make(){
82     shape a=start();
83     array<P,4> A={P(1),P(1),P(1),P(1)},B=A;
84     P one=ones(1);for(int l=0;l<4;l++)for(int i=0;i<4;i++)for(int j=0;j<4;j++){A[l][i,j]=a[i][l];B[l][i,j]=a[j][l
        ];}
85     for(auto m:{&A,&B})for(auto &q:*m){for(auto &x:q.b)x>=24;q.den=12;}
86     vector<P>L=inv(A),M=inv(B); // product polynomials bivar but L depends only a "real". dot
87     P hL=dot(h,L,one), hM=dot(h,M,one);
88     auto tensor=[](P const&w,P const&t){ // w only a, t only b. return S degree max
89         int d=max(w.d,t.d);P r(d); P x=lift(w,d),y=lift(t,d);
90         for(int i=0;i<S(d);i++)for(int j=0;j<S(d);j++)r(i,j)=x(i,0)*y(0,j);

```

```

91  r.den=x.den*y.den;return r;};
92  vector<R>&c=mode?g:f;
93  P K=-1*(tensor(hL,dot(c,M,one))+tensor(dot(c,L,one),hM));
94  if(!mode)K=K+L[1]+M[1]-divi(tensor(L[1],M[3])+tensor(L[3],M[1]),1000);// strict
95  P Taylor=K; dat r={a,a,24*K,{},{},24*K};int k=mode?5:1;r.k=k;
96  v idx={0,1,2,3};do{
97  array<P,4> U=A;
98  for(int l=0;l<4;l++)for(int i=0;i<4;i++)for(int j=0;j<4;j++)U[l](i,j)=4*a[i][l]*a[j][idx[l]];
99  for(auto&q:U){for(auto&x:q.b)x>=48;q.den=144;}
100  P z=U[0]+U[1]+U[2]+U[3];
101  vector<P> x=inv(U);
102  P N=mode?dot(h,x,z):-1*x[1];P D=z;while(D.d<k)D=D*z;
103  r.N.push_back(N);r.D.push_back(z);
104  P w=z-one; // 1/z  $\approx$  approx
105  P w2=w*w;
106  R m=mode?1:6;// need h>=-.5
107  P up=N+divi(m*(D*z),2),q=one -R(k)*w+R(k*(k+1)/2)*w2-R(k*(k+1)*(k+2)/6)*(w2*w)+R(k*(k+1)*(k+2)*(k+3)/24)*(w2*
    w2)-R(k*(k+1)*(k+2)*(k+3)*(k+4)/120)*(w2*w2*w);
108  r.t=r.t+up*q-divi(m*z,2);
109  }while(next_permutation(idx.begin(),idx.end()));
110  return r;
111 }
112 dat sub(dat const&a,bool y,int i,int j){
113 dat r={a.a,a.b,sub(a.K,y,i,j),{},{},a.t.b.empty()?a.t:sub(a.t,y,i,j),a.k};
114 V&b=y?r.b[i]:r.a[i];for(int h=0;h<4;h++)b[h]=(b[h]+(y?r.b[j][h]:r.a[j][h]))/2;
115 for(int n=0;n<a.N.size();n++){r.N.push_back(sub(a.N[n],y,i,j));r.D.push_back(sub(a.D[n],y,i,j));} return r;
116 }
117 int nodes=0,lp=0,lt=0,depth=0;
118
119 using Mat=vector<vector<R>>>;
120 map<pair<int,int>,Mat> T;
121 Mat const&ev(int x,int y){ if(T.count({x,y}))return T[{x,y}];
122 Mat m(S(y),vector<R>(S(x)));
123 for(int i=0;i<S(y);i++)for(int j=0;j<S(x);j++){R z=1;for(int k=0;k<4;k++)z*=Pow(RR(1[y][i][k]),1[x][j][k]).
    convert_to<R>());m[i][j]=z;}
124 return T[{x,y}]=m;}
125 Mat inverse(Mat Q){
126 // B/960 inverse of evaluation
127 int n=S(3); Mat B(n,vector<R>(n));
128 for(int j=0;j<n;j++){
129 map<v,R> p;p[v{0,0,0,0}]=960;
130 R den=1;
131 for(int h=0;h<4;h++)for(int m=0;m<1[3][j][h];m++){
132 map<v,R> b;
133 for(auto q:p)for(int t=0;t<4;t++)b[q.first+e(t)]+=q.second*(3*(t==h)-m);
134 p=b;den*=m+1;
135 }
136 for(auto q:p)B[id(3,q.first)][j]=q.second/den;
137 }
138 return B;}
139 P act(P const&a,Mat const&m,int d){
140 P r(d);int n=m.size(),h=S(a.d);
141 vector<RR>v(h);
142 for(int i=0;i<n;i++){ fill(v.begin(),v.end(),0);for(int j=0;j<h;j++){ R x=m[i][j];if(x!=0)for(int k=0;k<h;k
    ++v[k]+=a(j,k)*x;}
143 for(int j=0;j<n;j++)r(i,j)=inner_product(v.begin(),v.end(),m[j].begin(),RR(0));}
144 r.den=a.den; // evaluation factor caller
145 return r;}
146 const int degI=3;
147 bool testP(dat const&r){
148 int k=r.k; P F(degI);RR cost=0,Budget=RR(1)<<32;
149 static Mat I=inverse(ev(degI,degI));
150 for(int n=0;n<24;n++){
151 P const&d=r.D[n],&b=r.N[n];
152 P f=act(b,ev(k+1,degI),degI);f.den*=Pow(3,2*(k+1));
153 P M=act(d,ev(1,degI),degI);M.den*=9;
154 RR sc=Budget*Pow(M.den,k);

```

```

155 for(int i=0;i<f.b.size();i++)f.b[i]=M.b[i]==0?RR(0):RR((sc*f.b[i])/(f.den*pow(M.b[i],k))); // truncate
    towards zero
156 f.den=Budget;
157 f=act(f,I,degI); f.den*=960*960;
158 P o=f,w=ones(degI); for(int i=0;i<k;i++)o=o*d,w=w*d;
159 o=b-o;
160 // c approx ceil
161 RR c=0;RR fac=Budget*w.den;
162 RR big=o.den;
163 for(int i=0;i<o.b.size();i++)if(o.b[i]<0){assert(w.b[i]>0);
164 RR a=-o.b[i]*fac,v=w.b[i]*big;if(a>c*v)c=(a+v-1)/v;
165 }
166 cost+=c;F=F+f;
167 }
168 P bound=ones(0);bound.den=Budget;bound=cost*bound;
169 auto K=r.K+F-bound;
170 return *min_element(K.b.begin(),K.b.end())>=0;
171 }
172 bool uniform(shape const&a){for(auto&v:a)if(v[3]==3*(1<<24))return true;return false;}
173 bool goodT(P const&t){return *min_element(t.b.begin(),t.b.end())>=0;}
174 void run(dat r,int d){
175 nodes++;depth=max(depth,d);
176 if(nodes%100==0)cerr<<mode<<" "<<nodes<<" "<<d<<" "<<lp<<" "<<lt<<endl;
177 if(!r.t.b.empty()&&goodT(r.t)){lt++;return;}
178 if(!uniform(r.a)&&!uniform(r.b)){ if(testP(r)){lp++;return;} }
179 if(d>=24){cerr<<"fail "<<endl;for(auto a:{r.a,r.b}){for(auto q:a){for(R x:q)cerr<<x<<" ";cerr<<"| ";}cerr<<
    endl;}exit(1);}
180 // edge longest
181 using Z=tuple<R,bool,int,int>;Z z(-1,false,-1,-1);
182 for(bool y:{false,true}) for(int i=0;i<4;i++)for(int j=0;j<i;j++){shape&a=y?r.b:r.a;R w=0;for(int u=0;u<4;u++)
    w+=(a[i][u]-a[j][u])*(a[i][u]-a[j][u]);if(a[i][3]==3*(1<<24)||a[j][3]==3*(1<<24))w*=16;z=max(z,Z(w,y,i,j))
    ;}
183 auto [w,y,i,j]=z;run(sub(r,y,j,i),d+1);run(sub(r,y,i,j),d+1);
184 }
185 int main(int argc,char**argv){init();mode=argc>1;run(make(),0);cerr<<"done "<<nodes<<" "<<depth<<" lp="<<lp<<"
    lt="<<lt<<"\n";}

```

A.4 Scalar coefficients and symmetry

```

1 from fractions import Fraction as F
2 from itertools import permutations
3 import json
4
5 ZERO=(0,0,0,0)
6 def add(a,b):
7     c=dict(a)
8     for i,v in b.items(): c[i]=c.get(i,F(0))+v
9     return {i:v for i,v in c.items() if v}
10 def mul(a,b):
11     c={}
12     for i,u in a.items():
13         for j,v in b.items():
14             t=tuple(x+y for x,y in zip(i,j))
15             c[t]=c.get(t,F(0))+u*v
16     return {i:v for i,v in c.items() if v}
17 def scale(a,c): return {i:v*c for i,v in a.items() if v*c}
18 def power(a,k):
19     c={ZERO:F(1)}
20     for _ in range(k): c=mul(c,a)
21     return c
22 def indices(d):
23     for i in range(d+1):
24         for j in range(d-i+1):
25             for k in range(d-i-j+1):
26                 yield i,j,k,d-i-j-k
27 def linear(v):
28     return {tuple(int(i==j) for j in range(4)):F(a) for i,a in enumerate(v) if a}

```

```

29 W=linear([1,1,1,1])
30 X=[linear([1,1,F(1,3),0]),linear([1,0,F(1,3),0]),linear([1,0,F(-1,3),0])]
31 A=add(add(power(X[0],2),power(X[1],2)),power(X[2],2))
32 B=mul(mul(X[0],X[1]),X[2])
33 C=add(add(mul(power(X[0],2),power(X[1],2)),mul(power(X[0],2),power(X[2],2))),mul(power(X[1],2),power(X[2],2)))
34 basis=[{ZERO:F(1)},A,B,power(A,2),C,mul(A,B),power(A,3),mul(A,C),power(B,2)]
35 degrees=[0,2,3,4,4,5,6,6,6]
36 coeffs={
37   'H':[0,0,1000,-100,-150,300,-20,70,-510],
38   'F':[0,4000,-3000,700,-4000,-800,0,0,0],
39   'G':[1000,-1610,-3100,773,4000,-3980,0,0,0],
40 }
41 def homogenized(cs,degree):
42     r={}
43     for c,p,d in zip(cs,basis,degrees):
44         if c: r=add(r,scale(mul(p,power(W,degree-d)),F(c,1000)))
45     return r
46 tests=[('H+1/2',6,[500]+coeffs['H'][1:],F(1134673920,2985984000)),
47        ('F',5,coeffs['F'],F(0)),
48        ('G',14,coeffs['G'],F(21648384,248832000)),
49        ('(3-Euler)H',6,[c*(3-d) for c,d in zip(coeffs['H'],degrees)],F(0))]
50 results=[]
51 for name,degree,cs,expected in tests:
52     p=homogenized(cs,degree)
53     low=min(p.get(i,F(0)) for i in indices(degree))
54     at=[i for i in indices(degree) if p.get(i,F(0))==low]
55     assert low==expected,(name,low,expected)
56     results.append(dict(name=name,degree=degree,minimum=str(low),min_indices=at,coefficient_count=len(list(
57         indices(degree)))))
58 L=[[1,1,-1,-1],[1,-1,1,-1],[1,-1,-1,1]]
59 for perm in permutations(range(4)):
60     #  $T = L P L^t / 4$ :  $x$ -coordinate action of permuting  $p$ .
61     T=[[sum(F(L[i][k]*L[j][perm[k]],4) for k in range(4)) for j in range(3)] for i in range(3)]
62     assert all(sum(t!=0 for t in row)==1 for row in T)
63     assert all(sum(T[i][j]!=0 for i in range(3))==1 for j in range(3))
64     nonzero=[t for row in T for t in row if t]
65     assert all(abs(t)==1 for t in nonzero)
66     assert nonzero[0]*nonzero[1]*nonzero[2]==1
67 print(json.dumps({'small_checks':results,'all_24_symmetry_actions_verified':True},indent=2))

```